TRANSPORT ONE

APK Reverse-Engineering and Security Audit

APK	TransportOne.apk
Package	com.transportone.transport_one
Version	1.0.0 (versionCode 1)
Size	56,570,075 bytes (56.57 MB)
SHA-256	`08473df9fdf1656e9d4e1aa04b04d6620705ac438f66fccea5c7a210a5bf9fa4`
Audit date	19 October 2026

The APK contains multiple release-blocking issues. The most serious are compiled default credentials, an Android Debug signing certificate, client-side authorization, and sensitive operational photos/data stored unencrypted on the device.

Scope and method

This was a static, read-only review of the supplied APK. The review covered ZIP integrity, Android manifest, permissions, exported components, signing block and certificate metadata, Flutter AOT strings, embedded URLs and identifiers, native library hardening indicators, assets, local persistence markers, and architecture packaging. No production service was contacted and no account or live data was used.

Executive summary

Severity	Finding	Release decision
CRITICAL	Hardcoded admin password and universal driver PIN	Block release
HIGH	APK signed with Android Debug certificate	Block release
HIGH	Authorization and roles enforced locally	Block production use
HIGH	PII, trips and base64 photos stored in SharedPreferences	Fix before real data
HIGH	AI, real-time and external alert claims lack packaged implementation	Correct behavior and claims
MEDIUM	Cleartext HTTP explicitly permitted	Harden before release
MEDIUM	Google Maps API key embedded and reused in tile URLs	Restrict and rotate if needed
LOW	Universal APK, demo identifiers, versionCode 1 and system icon	Release hygiene

Detailed findings and solutions

CRITICAL

1. Hardcoded production credentials

Evidence

Flutter AOT strings contain the literal administrator password **admin123**, the message *Default password is: admin123*, a default driver PIN **123456**, and compiled administrator identities. These values are recoverable without running the app.

Risk

Anyone with the APK can learn the credentials. A shared default driver PIN also allows one compromised value to affect every newly created driver. Password rotation inside the UI cannot remove the original literals from already distributed APKs.

Required fix

- Remove every password, PIN, email allow-list and privileged identity from Flutter code and assets.
- Authenticate against a server using slow password hashing such as Argon2id or scrypt, per-user salts, rate limits and lockouts.
- Create unique one-time activation codes for drivers and require a change at first login.
- Invalidate all current credentials and sessions after deploying the server-side fix.

HIGH

2. Release package uses an Android Debug signing certificate

Evidence

The APK is not debuggable, but its APK Signature Scheme v2 signer certificate subject is **CN=Android Debug, O=Android, C=US**. Certificate SHA-256: **C431358BCAB014683BC83B975BBE3DF82FDFC752FC686EC0912EF5032AFD930A**. Only v2 signing was detected.

Risk

Debug keys are not suitable for production identity. Losing or replacing the key prevents trusted in-place updates, and anyone holding a matching debug keystore could publish an APK accepted as the same signer outside managed stores.

Required fix

- Generate a dedicated production upload/release key and store it in a protected secret manager or offline backup.
- Configure Gradle release signing explicitly and fail the build if the certificate subject contains Android Debug.
- Use Play App Signing when distributing through Google Play, and record certificate fingerprints for API restrictions.
- If this APK has already been distributed, plan a controlled signing migration where the distribution channel supports it.

HIGH

3. Authentication and role authorization are local and tamperable

Evidence

Compiled keys include **transport_one.is_logged_in**, **transport_one.auth_role**, **transport_one.current_role**, **transport_one.auth_email** and **transport_one.active_driver**. The app uses the SharedPreferences plugin and no first-party authentication/API endpoint was found. Privileged roles therefore appear to be decided entirely on the client.

Risk

On a rooted/debuggable environment, through backup/restore paths, instrumentation, or a modified APK, an attacker can change local role/session values and reach admin-only screens. Client checks cannot protect fleet records or privileged actions.

Required fix

- Move login, role assignment and every privileged operation to an authenticated backend.
- Use short-lived, server-signed sessions and verify authorization on every request.
- Treat local preferences as UI cache only; never use them as proof of identity or role.
- Add server-side audit logs and revoke sessions when a role, password or account status changes.

HIGH

4. Sensitive fleet data and photos are stored unencrypted

Evidence

The APK contains SharedPreferences keys for drivers, trips, alerts, activities, truck models and seatbelt records. Photo fields include **photoBase64** and **driverPhoto**. The manifest does not disable backup or define backup exclusion rules, so the Android default backup behavior applies.

Risk

Driver identifiers, phone/email values, routes, incident evidence and cabin photos may be exposed through device compromise, backup transfer, forensic extraction or accidental data copying. Base64 is encoding, not encryption. Large images in SharedPreferences can also cause slow startup, memory pressure and preference-file corruption.

Required fix

- Store structured records in an encrypted database and image files in encrypted app-private storage.
- Keep encryption keys in Android Keystore and avoid storing raw secrets alongside encrypted data.
- Set android:allowBackup="false" where business requirements permit, plus explicit dataExtractionRules/fullBackupContent exclusions.
- Define retention, deletion, consent and access-control rules for driver photos and location history.
- Upload only required evidence to a protected backend using authenticated HTTPS and delete expired local copies.

HIGH

5. AI and external alert behavior is not supported by packaged evidence

Evidence

The UI says *AI analyzing driver torso, seatbelt buckle & harness compliance* and states that an external safety notification was sent. However, the APK contains no ML model asset, TensorFlow Lite/ML Kit/MediaPipe dependency, inference endpoint, first-party alert endpoint, or push-notification implementation. Network URLs found are map tiles and the public OSRM routing service.

Risk

A photo may be labelled verified or rejected without a real detector, and users may believe an external safety team was notified when only a local record changed. In a transport-safety workflow this can create unsafe operational decisions and false assurance.

Required fix

- Implement a tested on-device model or authenticated server inference and record model confidence/version with each result.
- Use explicit states such as Pending Review, Verified by Model, and Verified by Human.
- Send alerts through a real backend queue with delivery receipts, retry handling and audit logs.
- Until implemented, remove AI, real-time and external notification claims from the UI.
- Do not automatically authorize a trip solely from an unvalidated photo result.

MEDIUM

6. Cleartext network traffic is explicitly allowed

Evidence

AndroidManifest.xml sets **android:usesCleartextTraffic="true"** and does not define a network security configuration. The currently extracted application URLs are HTTPS, but the manifest permits future or injected HTTP traffic.

Risk

Credentials, location or operational records could be exposed or modified if any code path uses HTTP. Broad cleartext permission also weakens protection against accidental endpoint regressions.

Required fix

- Set android:usesCleartextTraffic="false" for release builds.
- Add a Network Security Config that trusts system certificate authorities and permits no cleartext domains.
- Allow development hosts only in a debug-specific manifest overlay.
- Add automated tests that fail when release URLs are not HTTPS.

MEDIUM

7. Google Maps key is recoverable from manifest and Dart code

Evidence

A Google API key beginning **AIzaSy...M17PEY** appears in AndroidManifest.xml and four compiled map-tile URL templates. Client API keys are inherently extractable; the audit could not verify the Google Cloud restriction policy from the APK alone.

Risk

If the key is unrestricted, third parties can consume quota, create charges or cause service disruption. Reusing the key in raw tile URLs broadens exposure and makes Android package/certificate restrictions harder to apply correctly.

Required fix

- Restrict the key to the exact Android package and production signing certificate where the API supports Android restrictions.
- Restrict enabled APIs to only those required and configure budget/quota alerts.
- Use the official Google Maps SDK or a tile provider designed for authenticated client tile access.
- Rotate the key if logs show abuse or if it has ever been used without restrictions.

LOW

8. Release hygiene and packaging issues

Evidence

The APK bundles arm64-v8a, armeabi-v7a and x86_64 native libraries in one 56.57 MB file. It also contains demo-looking identifiers such as TRIP-IND-1001/TRIP-IND-1002 and sample vehicle numbers, uses versionCode 1, and references the Android system default application icon.

Risk

The universal package increases download size, demo data can be mistaken for real records, versionCode 1 provides little release history, and a system icon weakens product identity.

Required fix

- Publish an Android App Bundle to Google Play, or build split-per-ABI APKs for direct distribution.
- Move demo data behind an explicit development/demo flavor and ship production storage empty.
- Adopt monotonic version codes and documented release versioning.
- Provide adaptive launcher icons and verify branding on current Android versions.

Positive controls observed

- APK ZIP integrity check passed with no damaged entries.
- android:debuggable is not enabled in the manifest.
- Target SDK is 36 and minimum SDK is 24.
- Only the launcher activity is intentionally exported; other app/plugin activities and the location service are not exported.
- The exported ProfileInstaller receiver is protected by android.permission.DUMP.
- Extracted application URLs use HTTPS.
- The arm64 Flutter AOT library has a non-executable GNU stack and no writable-executable load segment.

Recommended remediation order

Priority	Action	Acceptance evidence
1	Remove compiled credentials and implement server authentication/authorization	Arbitrary local role edits cannot grant admin access; unique driver activation
2	Create production signing workflow	Release certificate is not Android Debug; fingerprint documented
3	Protect stored photos, PII and trip data	Encrypted storage, backup exclusions and retention tests
4	Implement or remove AI/external alert claims	Real inference tests and delivery receipts, or corrected UI
5	Disable cleartext and restrict Maps key	HTTPS-only release manifest and verified cloud restrictions
6	Clean production flavor and optimize artifacts	No demo records; AAB/split APKs; real icon/versioning

Verification checklist after fixes

- Search the release APK and confirm admin123, 123456 and personal admin identities are absent.
- Attempt login and role changes by modifying local preferences; server must reject unauthorized actions.
- Verify password hashing, rate limiting, lockout and session revocation on the backend.
- Confirm the release signer is the intended production certificate and preserve its backup.
- Verify Android backup/transfer does not include credentials, photos, routes or PII.
- Inspect the release manifest: cleartext disabled, debug disabled and only required components exported.
- Verify API-key restrictions in Google Cloud and monitor quota after rotation.
- Test actual seatbelt inference with positive/negative datasets, false-positive targets and human review.
- Test external alerts end to end, including offline retry and delivery failure.
- Run flutter analyze, unit/integration tests and a release build; then repeat static APK checks.

Technical evidence appendix

Area	Observed value
APK integrity	ZIP test passed; 116 entries
Android SDK	minSdk 24, targetSdk 36, compileSdk 36
Architectures	arm64-v8a, armeabi-v7a, x86_64
Signing	APK Signature Scheme v2; Android Debug certificate
Permissions	Internet, network state, coarse/fine location, camera
Network policy	usesCleartextTraffic=true; no networkSecurityConfig
Exported app component	MainActivity launcher; expected for app launch
Storage indicators	SharedPreferences keys for auth, roles, drivers, trips, alerts, activities, photos
Application URLs	HTTPS map tiles and public OSRM routing; no first-party backend found
ML implementation	No model asset or recognized mobile ML runtime found

Audit limitations

This report is based on static analysis of one supplied APK. It does not prove the absence of runtime-only vulnerabilities, server-side weaknesses, unrestricted cloud configuration, malicious dependencies, or issues triggered on a specific device. The APK was not installed, instrumented, or connected to production services. The v2 signer certificate and signing metadata were parsed; a complete independent cryptographic signature verification was not performed.

Release recommendation

Do not distribute this APK as a production fleet or safety application. Rebuild after resolving Findings 1-5, rotate exposed/default credentials, sign with a controlled production key, and repeat both static and device-based testing.